

Proposta de Domínio: Sistema de Agendamento de Quadras e Árbitros

1. Apresentação e Motivação

O projeto consiste em uma plataforma distribuída voltada para simplificar o agendamento de espaços esportivos e a contratação de profissionais de arbitragem. A motivação principal é resolver a fragmentação na organização de eventos esportivos amadores, integrando em um único ecossistema a busca por infraestrutura e o suporte técnico necessário para as partidas. O sistema adota uma arquitetura orientada a eventos (EDA) para garantir que a comunicação entre organizadores e prestadores ocorra de forma assíncrona e eficiente, utilizando um middleware de mensagens para notificações em tempo real.

2. Perfis de Usuário

O sistema é estruturado em torno de dois perfis de usuários fundamentais, atendendo aos requisitos de interação cliente-prestador:

- **Cliente (Organizador da Partida):** É o usuário que demanda o serviço. Ele utiliza o aplicativo para buscar quadras disponíveis, selecionar horários e solicitar a presença de árbitros para seus jogos.
- **Prestador de Serviços (Dono da Quadra / Árbitro):** É o usuário que oferta a infraestrutura ou a mão de obra técnica. Através de uma interface dedicada, ele recebe alertas de novas solicitações e gerencia sua agenda, podendo aceitar ou recusar demandas de forma instantânea.

3. Principais Funcionalidades

- **Criação de Agendamentos:** O cliente pode submeter solicitações especificando local, data e necessidade de arbitragem.
- **Notificações Orientadas a Eventos:** Assim que um agendamento é criado, o sistema publica um evento no middleware (MOM), notificando o prestador interessado sem a necessidade de atualizações manuais.
- **Gestão de Status:** O prestador pode atualizar o estado da reserva (ex: de "pendente" para "confirmado"), o que dispara uma notificação de retorno ao cliente.
- **Persistência e Consulta:** Histórico completo de agendamentos realizados e pendentes, garantindo a integridade dos dados de reserva.

4. Tecnologias Utilizadas

- **Backend:** Node.js com Express e TypeScript, seguindo princípios de Clean Architecture.
- **Banco de Dados:** PostgreSQL gerenciado via Drizzle ORM para persistência relacional.

- **Mensageria (MOM):** Redis Pub/Sub para a coordenação de eventos assíncronos.
- **Mobile:** Aplicativos desenvolvidos em Flutter para ambas as interfaces de usuário.

DIAGRAMA ARQUITETURAL:

